Programming Assignment 3

Running the project

Run **src\BayesNetwork\Main.java** and provide it with the input file as the first argument.

I am using the same syntax from the assignment.

Bayes' Network construction

First, we choose an ordering of the variables which is consistent with the topological sort.

In our case, this is the season variable, then the vertices' packages and lastly the edges' blockages.

```java
public class BayesNetwork {
    /*  A Bayes Network class.
     *      Contain a set of nodes Xi, one per variable (season, vertex, edges)
     *      A directed acyclic graph (DAG) sorted by a topologic order of influence.
     *
     *      Conditional distribution represented as a conditional probability table (CPT) giving the distribution over Xi,
     *      for each combination of parent values.
     *
     *
     *      Influence diagram:
     *
     *                     SEASON                    (Which sason is it)
     *                /  /   |   \   \
     *              V1  V2   V3  V4... Vn            (is the vertex contain a package)
     *              |  / |  / \  / \   /
     *              E1   E2    E3 ... Ek             (is the edge is blocked)
     */

```

Afterwards, we are creating a link between the season variable and all the vertices.

And from each edge to both of its ends (the vertices it is connecting).

Since this is the only probability influence in our system, its satisfies $P(X_i \,|Parents(X_i)) = P(X_i \,|X1, \dots, X_{i-1})$ which guarantees global semantics.

Reasoning algorithm

For the reasoning enumeration I have chose to use Influence by Enumeration algorithm taught in class (Ch. 14b page 5).

When enumerating the probability distribution over the domain of values for the variable of interest, we are going over all the possible combinations of values for the parents of said variable.

If a parent is already known (given in the evidence) we are using only its provided value with probability 1 (and the rest of the values is with probability 0).

Furthermore, we are also optimizing out search via removing barren nodes that have no effect on the calculation, and performing early-return when the probability of the current values of the parents is 0.

Bonus

I have done both bonuses. Shown in the running example below.

Bonus 1:

What is the probability that a certain path (set of edges) is free from blockages? (Note that the distributions of blockages in edges are NOT necessarily independent.) (bonus)

I have calculated the required probability with an iterative approach, for example:

For a path V1-V2-V3-V4

p(The path is not blocked) = P(V1-V2 is free | Evidence) * P(V2-V3 is free | Evidence, V1-V2 is free) *

P(V3-V4 is free | Evidence, V1-V2 and V2-V3 are free)

```java
/*  A path is blocked if any of the edges it contain is blocked.
 *  the distributions of blockages in edges are NOT necessarily independent, so we'll use conditional probability
 *
 *  Example:
 *      for a path V1-V2-V3-V4
 *      p(The path is not blocked) = P(V1-V2 is free | Evidence) * P(V2-V3 is free | Evidence, V1-V2 is free) *
 *          P(V3-V4 is free | Evidence, V1-V2 and V2-V3 are free)
 */
private void reasonCertainPath(String[] tokens){
    if(tokens.length <= 4 ){
        System.out.println(x:"A path must consist of at least two vertices!");
        return;
    }

    List<Integer> indices = getIndicesFromTokens(tokens);
    double pathNotBlockedProbability = calculateFreePathProbability(indices);

    System.out.println("The given path is free with probability of " + pathNotBlockedProbability + " given the evidence.\n")
}
```

Bonus 2:

What is the path from a given location to a goal that has the highest probability of being free from blockages?

I've found all the simple paths from start to goal (simple = not returning to the same vertex twice) using a DFS with backtracking.

After that, I have used the function from bonus 1 to evaluate each path, and kept the one with the highest probability of being free.

```java
private void reasonBestPath(String[] tokens){
    int startIndex = Integer.parseInt(tokens[2].substring(beginIndex:1));
    int goalIndex = Integer.parseInt(tokens[3].substring(beginIndex:1));

    List<List<Integer>> allPathsList = findAllSimplePaths(startIndex, goalIndex);

    double maxProbabilityOfBeingFreePath = -1.0;
    List<Integer> maxProbablityPath = null;

    for(List<Integer> curr : allPathsList){
        double currPathisFreeProbability = calculateFreePathProbability(curr);

        if(currPathisFreeProbability > maxProbabilityOfBeingFreePath){
            maxProbabilityOfBeingFreePath = currPathisFreeProbability;
            maxProbablityPath = curr;
        }
    }

    if(maxProbablityPath == null){
        System.out.println("There is no path between V" + startIndex +" and V" + goalIndex + " \n");
    } else {
        System.out.println(x:"The path with the highest probability of being free is:\n");

        boolean isFirst = true;
        for(Integer i :maxProbablityPath){
            if(isFirst){
                isFirst = false;
                System.out.print("V" + i);
            } else {
                System.out.print(" -> V" + i);
            }
        }
        System.out.println("\nWith a probability of " + maxProbabilityOfBeingFreePath);
    }
}
```

Running examples

Example 1:

Input:

#N 5 ; number of vertices N in graph (from V1 to Vn)

#E1 1 2 W1 ; Edge 1 from vertex 1 to vertex 2, weight 1

#E2 3 4 W1 ; Edge 2 from vertex 3 to vertex 4, weight 1

#E3 2 3 W2 ; Edge 3 from vertex 2 to vertex 3, weight 2

#E4 1 3 W1 ; Edge 4 from vertex 1 to vertex 3, weight 4

#E5 3 5 W2 ; Edge 6 from vertex 4 to vertex 5, weight 2

#E6 1 5 W2 ; Edge 7 from vertex 1 to vertex 5, weight 2

#P1 V2 0.1 ; Vertex V2 probability of package given low demand season 0.1

#P2 V3 0.22 ; Vertex V3 probability of package given low demand season 0.22

#F V1 V2 ; Edge from V1 to V2 is fragile

#F V2 V3 ; Edge from V2 to V3 is fragile

#L 0.11 ; Global leakage probability 0.11

#Q 0.2 ; Parameter Q is 0.2

#S 0.1 0.4 0.5 ; Prior distribution over season: 0.1 for low, 0.4 medium, 0.5 high

Output:

User input is marked

Program output is unmarked

--

starting...

==============   Bayes Network   ==============

Global Leakage L: 0.11

Distribution parameter Q: 0.2

*** SEASON: ***

    P(low | evidence) = 0.1

    P(medium | evidence) = 0.4

    P(high | evidence) = 0.5

*** VERTECIES: ***

    Vertex V2

        P(package | low) = 0.1

        P(package | medium) = 0.2

        P(package | high) = 0.4

    Vertex V3

        P(package | low) = 0.22

        P(package | medium) = 0.44

        P(package | high) = 0.88

*** Edges: ***

    Edge V1, V2:

        P(blocked | no package at V1, no package at V2) = 0.11

P(blocked | no package at V1, package at V2) = 0.8

P(blocked | package at V1, no package at V2) = 0.8

P(blocked | package at V1, package at V2) = 0.96

Edge V2, V3:

P(blocked | no package at V2, no package at V3) = 0.11

P(blocked | no package at V2, package at V3) = 0.6

P(blocked | package at V2, no package at V3) = 0.6

P(blocked | package at V2, package at V3) = 0.84

================ Interactive Querying ================

Supported operations:

1) Reset - reset the evidence
2) Add - add a piece of evidence to the evidence list
3) Reason - perform probabilistic reasoning on the evidence
4) Quit - exit the program

Please choose an option

add package exist V2

reason season

*** SEASON: ***

P(low | evidence) = 0.034482758620689655

P(medium | evidence) = 0.27586206896551724

P(high | evidence) = 0.689655172413793

reason edges

*** Edges: ***

Edge V1, V2:

P(blocked | evidence) = 0.8

P(not blocked | evidence) = 0.19999999999999993

Edge V2, V3:

P(blocked | evidence) = 0.776606896551724

P(not blocked | evidence) = 0.22339310344827587

add blockage absent V1 V2

reason edges

*** Edges: ***

Edge V1, V2:

P(blocked | evidence) = 0.0

P(not blocked | evidence) = 1.0

Edge V2, V3:

P(blocked | evidence) = 0.776606896551724

P(not blocked | evidence) = 0.2233931034482759

reset

reason edges

*** Edges: ***

Edge V1, V2:

P(blocked | evidence) = 0.31010000000000004

P(not blocked | evidence) = 0.6899000000000001

Edge V2, V3:

P(blocked | evidence) = 0.51137

P(not blocked | evidence) = 0.4886300000000001

reason path V1 V2 V3 V4

The given path is free with probability of 0.39017974000000005 given the evidence.

reason bestPath V5 V2

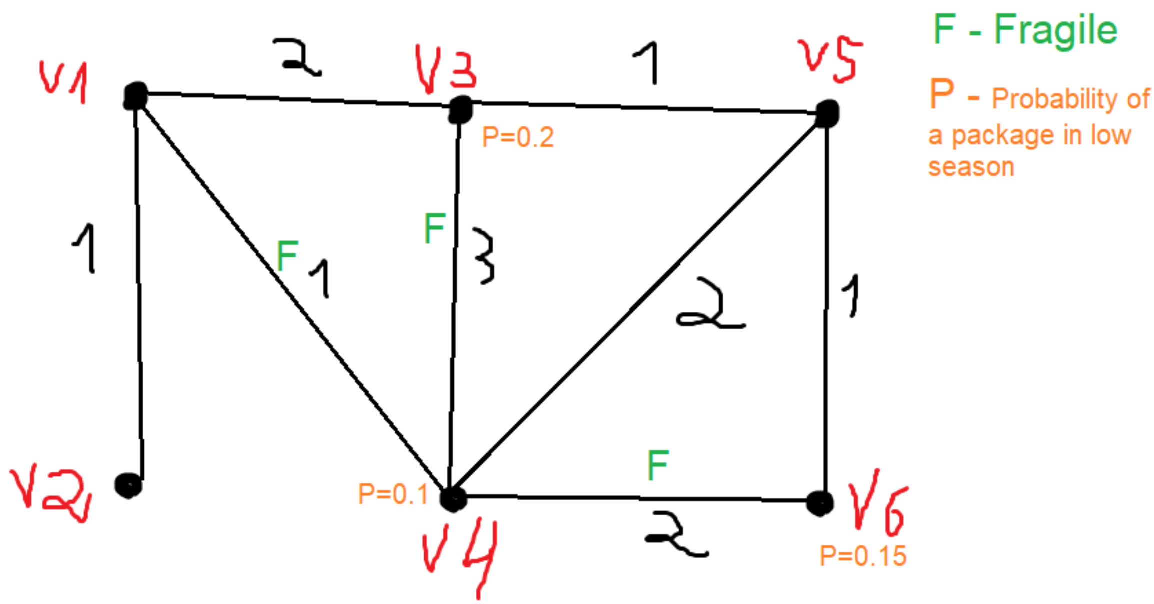
The path with the highest probability of being free is:

V5 -> V1 -> V2

With a probability of 0.6899000000000001

quit

Example 2:



Input

#N 6 ; number of vertices N in graph (from V1 to V6)

#E1 1 2 W1

#E2 1 3 W2

#E3 1 4 W1

#E4 3 4 W3

#E5 3 5 W1

#E6 4 5 W2

#E7 4 6 W2

#E8 5 6 W1

#P1 V3 0.2

#P2 V4 0.1

#P3 V6 0.15

#F V1 V4

#F V3 V4

#F V4 V6

#L 0.06   ; Global leakage probability 0.06

#Q 0.25   ; Parameter Q is 0.25

#S 0.2 0.7 0.1 ; Prior distribution over season: 0.2 for low, 0.7 medium, 0.1 high

Output:

starting...

============== Bayes Network ==============

Global Leakage L: 0.06

Distribution parameter Q: 0.25

*** SEASON: ***

P(low | evidence) = 0.2

P(medium | evidence) = 0.7

P(high | evidence) = 0.1

*** VERTECIES: ***

Vertex V3

P(package | low) = 0.2

P(package | medium) = 0.4

P(package | high) = 0.8

Vertex V4

P(package | low) = 0.1

P(package | medium) = 0.2

P(package | high) = 0.4

Vertex V6

P(package | low) = 0.15

P(package | medium) = 0.3

P(package | high) = 0.6

*** Edges: ***

Edge V1, V4:

P(blocked | no package at V1, no package at V4) = 0.06
P(blocked | no package at V1, package at V4) = 0.75
P(blocked | package at V1, no package at V4) = 0.75
P(blocked | package at V1, package at V4) = 0.9375

Edge V3, V4:

P(blocked | no package at V3, no package at V4) = 0.06
P(blocked | no package at V3, package at V4) = 0.25
P(blocked | package at V3, no package at V4) = 0.25
P(blocked | package at V3, package at V4) = 0.4375

Edge V4, V6:

P(blocked | no package at V4, no package at V6) = 0.06
P(blocked | no package at V4, package at V6) = 0.5
P(blocked | package at V4, no package at V6) = 0.5
P(blocked | package at V4, package at V6) = 0.75

================ Interactive Querying ================

Supported operations:

1) Reset - reset the evidence
2) Add - add a piece of evidence to the evidence list
3) Reason - perform probabilistic reasoning on the evidence
4) Quit - exit the program

Please choose an option

add package absent V3
reason edges

*** Edges: ***

Edge V1, V4:

P(blocked | evidence) = 0.18420000000000003

P(not blocked | evidence) = 0.8158

Edge V3, V4:

P(blocked | evidence) = 0.0942

P(not blocked | evidence) = 0.9057999999999999

Edge V4, V6:

P(blocked | evidence) = 0.24774000000000004

P(not blocked | evidence) = 0.75226

reason season

*** SEASON: ***

P(low | evidence) = 0.26666666666666666

P(medium | evidence) = 0.6999999999999998

P(high | evidence) = 0.033333333333333326

add package exist V6

reason vertices

*** Vertices Distribution: ***

Vertex V1

P(package | evidence) = 0.0

P(no package | evidence) = 1.0

Vertex V2

P(package | evidence) = 0.0

P(no package | evidence) = 1.0

Vertex V3

P(package | evidence) = 0.0

P(no package | evidence) = 1.0

Vertex V4

P(package | evidence) = 0.2

P(no package | evidence) = 0.8

Vertex V5

P(package | evidence) = 0.0

P(no package | evidence) = 1.0

Vertex V6

P(package | evidence) = 1.0

P(no package | evidence) = 0.0

reason path V2 V1 V3 V5 V6

The given path is free with probability of 1.0 given the evidence.

reason path V2 V1 V4 V6

The given path is free with probability of 0.3885 given the evidence.

reason season

*** SEASON: ***

P(low | evidence) = 0.14814814814814817

P(medium | evidence) = 0.7777777777777779

P(high | evidence) = 0.07407407407407407 reset

reason season

*** SEASON: ***

P(low | evidence) = 0.20000000000000004

P(medium | evidence) = 0.7000000000000001

P(high | evidence) = 0.10000000000000002

quit